



Design Patterns and Unit Testing

Names:	Victor Jerald Balagan Benedict Wrian Cajudo Travis Joe Carlos John Kyrant Mendoza	Date:	09/12/26
Program:	Computer Engineering		

A. Learning Outcomes:

At the end of this laboratory exercise, students should be able to:

- Apply software design patterns in application development.
- Implement the Singleton and Factory Design Patterns.
- Create unit tests using a testing framework.
- Validate software functionality through automated testing.
- Collaborate effectively in a team-based software development project.

B. Scenario:

Paws and Care Veterinary Clinic currently manages pet appointments manually, causing scheduling conflicts and difficulty tracking records. To improve efficiency, the clinic wants a Vet Clinic Appointment Management System that allows staff to register pet owners, manage pet records, schedule appointments, and cancel bookings. The system will utilize the Singleton Pattern for managing a single clinic database, the Factory Pattern for creating different pet types, and Unit Testing to ensure all functionalities work correctly.

C. Functional Requirements

1. Pet Owner Management
 - Register new pet owners.
 - View registered pet owners.
 - Store owner information (Owner ID, Name, Contact Number).
 2. Pet Management
 - Add pet records.
 - View pet information.
 - Associate pets with their respective owners.
 - Support different pet types:
 - Dog
 - Cat
 - Bird
 - Rabbit
 3. Appointment Management
 - Schedule appointments.
 - View appointment schedules.
 - Cancel appointments.
 - Update appointment status (Scheduled, Completed, Cancelled).
 4. Design Pattern Implementation
 - Singleton Pattern
 - Create a single ClinicDatabase instance to manage all records.
 - Factory Pattern
 - Create pet objects through a PetFactory class.
 5. Unit Testing
- Develop unit tests for:
- Register Pet Owner
 - Add Pet Record
 - Schedule Appointment
 - Cancel Appointment



Laboratory Report 4

Design Patterns and Unit Testing

- Validate Singleton Instance

(Please include the following: **1.** Screenshots of the code **2.** Screenshots of the output)

Duplicate if necessary

Screenshots of Code (Name of Members) – Balagan, Cajudo, Carlos, Mendoza

```
Code Blame 212 lines (180 loc) · 8.2 KB
1 import sys
2 import unittest
3
4 # --- 1-3: MODULE IMPORTS ---
5 # Importing the existing modules from the 'modules' directory.
6 # Since the exact function signatures in your 1-3 modules are hidden,
7 # these assume standard procedural signatures passing the database instance.
8 try:
9     from modules.pet_owner_management import register_owner, view_owners
10    from modules.appointment import schedule_appointment, view_appointments, cancel_appointment, update_appointment_status
11    # If pet management logic (viewing/associating) is in clinic_database:
12    # from modules.clinic_database import view_pets
13 except ImportError as e:
14     print(f"Warning: Ensure your modules are correctly set up in the 'modules' folder. Missing: {e}")
15
16 # --- 4. DESIGN PATTERN IMPLEMENTATION ---
17
18 # Singleton Pattern: ClinicDatabase
19 class ClinicDatabase:
20     _instance = None
21
22     def __new__(cls):
23         if cls._instance is None:
24             cls._instance = super(ClinicDatabase, cls).__new__(cls)
25             # Centralized storage for the clinic
26             cls._instance.owners = {} # {owner_id: {"name": name, "contact": contact}}
27             cls._instance.pets = [] # List of Pet objects
28             cls._instance.appointments = {} # {apt_id: {"pet_name": name, "status": status}}
29             return cls._instance
30
31 # Factory Pattern: PetFactory
32 class Pet:
33     def __init__(self, name, owner_id):
34         self.name = name
35
36         self.owner_id = owner_id
37
38     class Dog(Pet): pass
39     class Cat(Pet): pass
40     class Bird(Pet): pass
41     class Rabbit(Pet): pass
42
43 class PetFactory:
44     @staticmethod
45     def create_pet(pet_type, name, owner_id):
46         pet_type = pet_type.lower()
47         if pet_type == 'dog':
48             return Dog(name, owner_id)
49         elif pet_type == 'cat':
50             return Cat(name, owner_id)
51         elif pet_type == 'bird':
52             return Bird(name, owner_id)
53         elif pet_type == 'rabbit':
54             return Rabbit(name, owner_id)
55         else:
56             raise ValueError(f"Pet type '{pet_type}' is not supported.")
57
58 # --- 5. UNIT TESTING ---
59
60 class TestClinicSystem(unittest.TestCase):
61     def setUp(self):
62         # Reset the singleton instance before each test to ensure an isolated state
63         ClinicDatabase._instance = None
64         self.db = ClinicDatabase()
65
66     def test_validate_singleton_instance(self):
67         db1 = ClinicDatabase()
```



Laboratory Report 4

Design Patterns and Unit Testing

```
68         db2 = ClinicDatabase()
69         self.assertIs(db1, db2, "ClinicDatabase is not a Singleton! Instances differ.")
70
71     def test_register_pet_owner(self):
72         owner_id = "01"
73         self.db.owners[owner_id] = {"name": "John Doe", "contact": "1234567890"}
74         self.assertIn(owner_id, self.db.owners)
75         self.assertEqual(self.db.owners[owner_id]["name"], "John Doe")
76
77     def test_add_pet_record(self):
78         pet = PetFactory.create_pet("Dog", "Buddy", "01")
79         self.db.pets.append(pet)
80         self.assertEqual(len(self.db.pets), 1)
81         self.assertIsInstance(self.db.pets[0], Dog)
82         self.assertEqual(self.db.pets[0].name, "Buddy")
83
84     def test_schedule_appointment(self):
85         apt_id = "A1"
86         self.db.appointments[apt_id] = {"pet_name": "Buddy", "status": "Scheduled"}
87         self.assertIn(apt_id, self.db.appointments)
88         self.assertEqual(self.db.appointments[apt_id]["status"], "Scheduled")
89
90     def test_cancel_appointment(self):
91         apt_id = "A1"
92         self.db.appointments[apt_id] = {"pet_name": "Buddy", "status": "Scheduled"}
93         self.db.appointments[apt_id]["status"] = "Cancelled"
94         self.assertEqual(self.db.appointments[apt_id]["status"], "Cancelled")
95
96
97     # --- TERMINAL APP (CLI) ---
98
99     def run_tests():
100         print("\n--- Running Unit Tests ---")
101         # Pass sys.argv to avoid breaking the CLI loop when unittest runs
102         unittest.main(argv=['first-arg-is-ignored'], exit=False)
103         print("-----\n")
104
105     def main():
106         db = ClinicDatabase()
107
108         while True:
109             print("\n== Veterinary Clinic Management System ==")
110             print("1. Pet Owner Management")
111             print("2. Pet Management (Factory implementation)")
112             print("3. Appointment Management")
113             print("4. Run Unit Tests")
114             print("5. Exit")
115
116             choice = input("Enter your choice: ")
117
118             if choice == '1':
119                 print("\n--- Pet Owner Management ---")
120                 print("a. Register Owner")
121                 print("b. View Owners")
122                 sub_choice = input("Select option: ").lower()
123
124                 if sub_choice == 'a':
125                     owner_id = input("Enter Owner ID: ")
126                     name = input("Enter Owner Name: ")
127                     contact = input("Enter Contact Number: ")
128                     # Assuming your module 1 function takes these params
129                     try:
130                         register_owner(db, owner_id, name, contact)
131                         print(f"Owner {name} registered successfully!")
132                     except NameError:
133                         db.owners[owner_id] = {"name": name, "contact": contact}
134                         print(f"Owner {name} registered directly via CLI.")
135                 elif sub_choice == 'b':
136                     try:
137                         view_owners(db)
138                     except NameError:
139                         print("Registered Owners:", db.owners)
140
141             elif choice == '2':
142                 print("\n--- Pet Management ---")
143                 print("a. Add Pet Record")
144                 print("b. View Pets")
145                 sub_choice = input("Select option: ").lower()
146
147                 if sub_choice == 'a':
148                     try:
149                         p_type = input("Enter pet type (Dog, Cat, Bird, Rabbit): ")
150                         p_name = input("Enter pet name: ")
151                         o_id = input("Enter owner ID: ")
152
153                         # Core requirement: Create objects through PetFactory
154                         pet = PetFactory.create_pet(p_type, p_name, o_id)
155                         db.pets.append(pet)
156                         print(f"Successfully created and associated a {pet.__class__.__name__} named '{pet.name}'!")
157                     except ValueError as e:
158                         print(f"Error: {e}")
159                 elif sub_choice == 'b':
160                     for p in db.pets:
```

Laboratory Report 4

Design Patterns and Unit Testing

```
161         print(f"- Type: {p.__class__.__name__}, Name: {p.name}, Owner ID: {p.owner_id}")
162     if not db.pets:
163         print("No pets registered yet.")
164
165     elif choice == '3':
166         print("\n--- Appointment Management ---")
167         print("a. Schedule Appointment")
168         print("b. View Appointments")
169         print("c. Update/Cancel Appointment")
170         sub_choice = input("Select option: ").lower()
171
172     if sub_choice == 'a':
173         apt_id = input("Enter Appointment ID: ")
174         pet_name = input("Enter Pet Name: ")
175         try:
176             schedule_appointment(db, apt_id, pet_name)
177             print("Appointment scheduled successfully!")
178         except NameError:
179             db.appointments[apt_id] = {"pet_name": pet_name, "status": "Scheduled"}
180             print("Appointment scheduled directly via CLI (Module not fully linked).")
181     elif sub_choice == 'b':
182         try:
183             view_appointments(db)
184         except NameError:
185             print("Appointments:", db.appointments)
186     elif sub_choice == 'c':
187         apt_id = input("Enter Appointment ID to modify: ")
188         new_status = input("Enter new status (Scheduled, Completed, Cancelled): ").capitalize()
189         try:
190             if new_status == "Cancelled":
191                 cancel_appointment(db, apt_id)
192             else:
193                 update_appointment_status(db, apt_id, new_status)
194         except NameError:
195             if apt_id in db.appointments:
196                 db.appointments[apt_id]["status"] = new_status
197                 print(f"Appointment {apt_id} status updated to {new_status}.")
198             else:
199                 print("Appointment not found.")
200
201     elif choice == '4':
202         run_tests()
203
204     elif choice == '5':
205         print("Exiting system. Goodbye!")
206         sys.exit(0)
207
208     else:
209         print("Invalid choice. Please enter a number from 1 to 5.")
210
211 if __name__ == "__main__":
212     main()
```

Screenshots of Output

Design Patterns and Unit Testing

```
=== Veterinary Clinic Management System ===
1. Pet Owner Management
2. Pet Management (Factory implementation)
3. Appointment Management
4. Run Unit Tests
5. Exit
Enter your choice: 1

--- Pet Owner Management ---
a. Register Owner
b. View Owners
Select option: a
Enter Owner ID: 56
Enter Owner Name: joe
Enter Contact Number: 099874556
Owner joe registered directly via CLI (Module not fully linked).

=== Veterinary Clinic Management System ===
1. Pet Owner Management
2. Pet Management (Factory implementation)
3. Appointment Management
4. Run Unit Tests
5. Exit
Enter your choice: 1

--- Pet Owner Management ---
a. Register Owner
b. View Owners
Select option: b
Registered Owners: {'56': {'name': 'joe', 'contact': '099874556'}}

=== Veterinary Clinic Management System ===
1. Pet Owner Management
2. Pet Management (Factory implementation)
3. Appointment Management
4. Run Unit Tests
5. Exit
Enter your choice:

--- Pet Management ---
a. Add Pet Record
b. View Pets
Select option: a
Enter pet type (Dog, Cat, Bird, Rabbit): Dog
Enter pet name: Joe
Enter owner ID: 123
Successfully created and associated a Dog named 'Joe'!

=== Veterinary Clinic Management System ===
1. Pet Owner Management
2. Pet Management (Factory implementation)
3. Appointment Management
4. Run Unit Tests
5. Exit
Enter your choice: 3

--- Appointment Management ---
a. Schedule Appointment
b. View Appointments
c. Update/Cancel Appointment
Select option: b
Appointments: {'123': {'pet_name': 'Joe', 'status': 'Scheduled'}}
```



Design Patterns and Unit Testing

```
=== Veterinary Clinic Management System ===
1. Pet Owner Management
2. Pet Management (Factory implementation)
3. Appointment Management
4. Run Unit Tests
5. Exit
Enter your choice: 4

--- Running Unit Tests ---
.....
-----
Ran 5 tests in 0.007s

OK
-----
```



Design Patterns and Unit Testing

D. Findings, Observations, and Comments:

Guide Questions:

1. What are your Group's realizations in this laboratory?
2. What are the advantages of using design patterns in software development?
3. What challenges did you face during testing phases?

What are your Group's realizations in this laboratory? Our group realized that managing centralized clinic data requires a reliable architectural structure to prevent conflicting application states. Using the Singleton Pattern for the ClinicDatabase ensured all modules share a single source of truth, while the Factory Pattern (PetFactory) streamlined pet object creation, keeping the main program logic clean and organized.

What are the advantages of using design patterns in software development? Design patterns provide standard, reusable solutions that improve code maintainability, modularity, and scalability. The Singleton pattern strictly controlled class instantiation for the centralized database, and the Factory pattern promoted loose coupling. This makes it easier to add new pet types in the future without altering the core scheduling logic.

What challenges did you face during testing phases? The main challenge was maintaining test isolation when validating the Singleton class. Because a Singleton persists state globally, data injected during one test would carry over to the next, causing unexpected behavior. We resolved this by using the setUp method in the unittest framework to explicitly reset ClinicDatabase._instance to None before each test, ensuring a fresh database state.

PICTURE/PROOF OF DOING THE ACTIVITY

Laboratory Report 4
Design Patterns and Unit Testing



Laboratory Report 4
Design Patterns and Unit Testing



Laboratory Report 4
Design Patterns and Unit Testing



Design Patterns and Unit Testing

--



Laboratory Report 4

Design Patterns and Unit Testing

E. Score Sheet

Criteria	Poor (25%)	Fair (50%)	Good (75%)	Excellent (100%)	Score
I. Completeness, Documentation, and Organization of Report	The report was incomplete, messy, and had many unanswered questions.	The report was complete, messy, and had many unanswered questions.	The report was complete, neat, and had 1 or 2 unanswered questions.	The report was complete, neat, and all the questions had been answered.	
II. Coding Design and Patterns	The program had inappropriate elements and structures, incorrect indentation, and poor program documentation.	The program had corrected indention, but inappropriate elements and structures and poor program documentation.	The program had corrected indention, adequate program documentation, but inappropriate elements and structures.	The program had appropriate elements and structures, correct indention, and excellent program documentation.	
III. Findings, Observations, and Comments	The findings. observations, and comments were not based on the gathered data and results. All thoughts were inconsistent and unclear.	The findings. observations, and comments based on the gathered data and results but were not elaborated. Not all thoughts were consistent and clear.	The findings. observations, and comments were based on the gathered data and results and were mostly elaborated. Most of the thoughts were consistent and clear.	The findings. observations, and comments were based on the gathered data and results and were completely elaborated. All the thoughts were consistent and clear.	
IV. Grammar and Composition	The words used were inappropriate, wrong grammar usage, and poor sentence construction was observed.	The words used were appropriate; however, bad grammar and poor sentence construction were observed.	The words used were appropriate, had proper grammar usage, but poor sentence construction was observed.	The words used were appropriate, had proper grammar usage, and excellent sentence construction.	
SCORE:					

CHECKED			
Rating		Signature	
		Date	